Vue 3 Fundamentos: Del Componente al DOM

Guía visual progresiva para el desarrollo Front-end (CDN + Composition API).

// Componentes | Props | v-for | JSON | Ciclo de Vida | Estilos

NotebookLM

Código Tradicional

```html
<div class='card'>
  <p>Cita 1</p>
</div>

<div class='card'>
  <p>Cita 2</p>
</div>

<div class='card'>
  <p>Cita 3</p>
</div>
```

Repetitivo. Si cambias el diseño de una tarjeta, debes cambiar todas manualmente.

Enfoque Basado en Componentes

```html
<quote-item></quote-item>
<quote-item></quote-item>
<quote-item></quote-item>
```

Reutilizable. Modificas el componente en un solo lugar y se actualiza en toda la aplicación.

Un componente es una pieza de interfaz de usuario (UI) independiente, encapsulada y reutilizable que agrupa HTML, lógica y estilos.

NotebookLM

JavaScript

```javascript
const app = Vue.createApp({ setup() { return {} } });

app.component('quote-item', {
  template: `<div class='card'>
              <p>"La web es el lienzo."</p>
            </div>`
});
app.mount('#app');
```

Registra el componente de forma global.

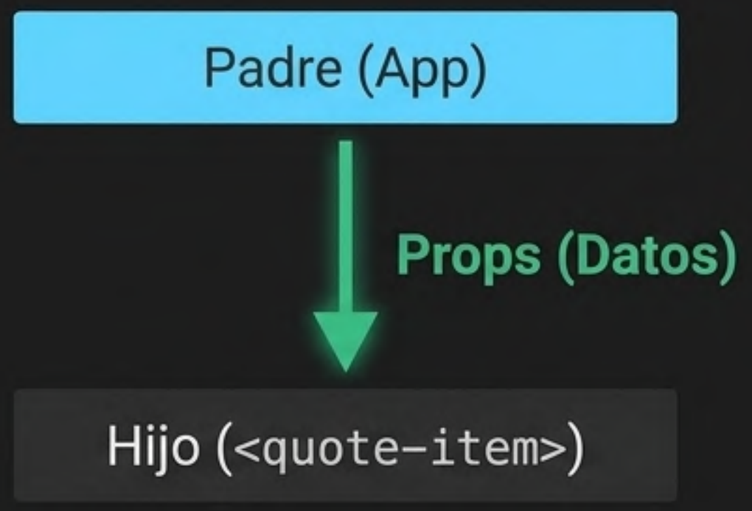
Define el nombre de nuestra nueva etiqueta HTML personalizada.

El bloque de HTML que se inyectará en el DOM cada vez que usemos la etiqueta.

HTML

```html
<div id='app'>
  <quote-item></quote-item>
</div>
```

NotebookLM



Las *props* son variables personalizadas que viajan hacia abajo: del componente padre al componente hijo.

JavaScript

```javascript
app.component('quote-item', {
  props: {
    text: { type: String, required: true },
    author: { type: String, required: true }
  },
  template: `<div class='card'>
              <p>"{{ text }}"</p>
              <small>- {{ author }}</small>
            </div>`
});
```

Define el 'contrato' de datos que el hijo espera recibir.

Validación estricta. Vue lanzará advertencias en consola si falta el dato o el tipo es incorrecto.

Interpolación. Imprime el valor de la prop directamente en el HTML.

NotebookLM

¿Cómo inyectamos la información en nuestro componente personalizado?
Los nombres de las props se convierten en atributos HTML.

HTML

```html
<div id='app'>
  <!-- Pasando valores estáticos a través de props -->
  <quote-item
    text="Aprender Vue es como armar legos."
    author="Estudiante Anónimo">
  </quote-item>
</div>
```

El componente padre envía el String estático. El componente hijo lo recibe y lo renderiza.

Nota: Hasta aquí los datos son fijos. El siguiente paso es renderizar listas dinámicas.

NotebookLM

`v-for` es una directiva que itera sobre un array y renderiza un componente por cada elemento de la lista.

HTML

```html
<div id='app'>
<div id='app'>
  <quote-item v-for="item in quotesList"
    :key="item.id"
    :text="item.quote"
    :author="item.author"
  </quote-item>
</div>
```

Crea un bucle. Genera un `<quote-item>` por cada objeto en `quotesList`.

CRUCIAL. Vue necesita un identificador único para saber qué elemento del DOM actualizar si la lista cambia.

Los dos puntos (:) significan *v-bind*. Conectan la prop con el valor dinámico del objeto actual (`item.quote`).

NotebookLM

Centralizar datos significa separar la información cruda de la lógica visual. **Movemos nuestros datos** a una fuente externa.

quotes.json

```
[
  {
    "id": 1,
    "quote": "El código es poesía.",
    "author": "WordPress"
  },
  {
    "id": 2,
    "quote": "Vue hace lo simple, hermoso.",
    "author": "Developer"
  }
]
```

- Estructura pura de Array con Objetos.
- Cada objeto contiene un `id` único (ideal para nuestro `:key`).
- Los campos `quote` y `author` mapean directamente con nuestras *props*.

NotebookLM

[Creación] — **[onMounted (Montado en el DOM)]** — [Actualización]

`onMounted` es un hook que ejecuta código exactamente cuando la app se inserta en el navegador. Es el lugar ideal para hacer peticiones (fetch).

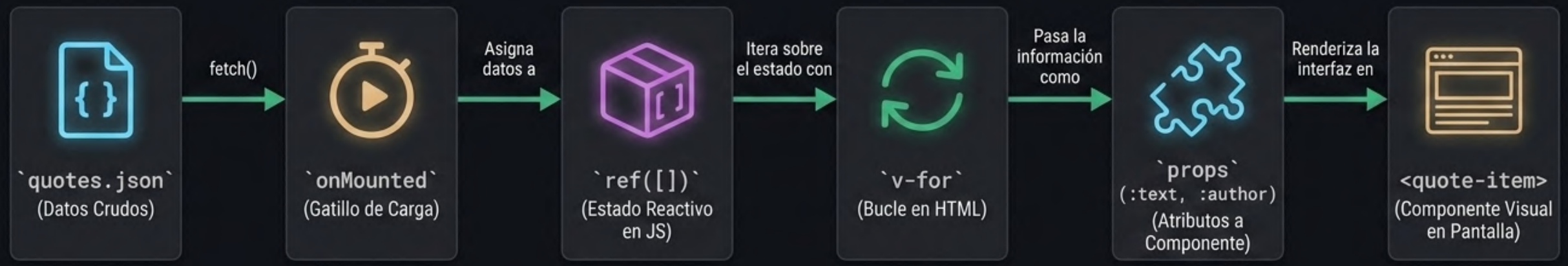
JavaScript Composition API

```
1   const { ref, onMounted } = Vue;
2
3   setup() {
4     const quotesList = ref([]); // 1. Estado reactivo vacío
5
6     onMounted(async () => {
7       const response = await fetch('quotes.json'); // 2. Petición HTTP
8       quotesList.value = await response.json();    // 3. Guardar datos
9     });
10
11    return { quotesList }; // 4. Exponer al HTML
12  }
```

Variable reactiva. Si su valor cambia, Vue redibuja el HTML automáticamente.

Para modificar el contenido de un `ref` dentro del bloque `<script>`, siempre debemos usar `.value`.

NotebookLM



La arquitectura unidireccional de Vue: los datos siempre fluyen hacia abajo, desde el estado hasta la vista.

NotebookLM

Todo Junto (Parte 1): La Vista

index.html

```html
<!-- Importamos Vue 3 vía CDN -->
<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>

<div id="app">
   <h1>Citas Inspiradoras</h1>

   <div class="grid-container">
      <!-- Componente dinámico impulsado por datos -->
      <quote-item
         v-for="item in quotesList"
         :key="item.id"
         :text="item.quote"
         :author="item.author">
      </quote-item>
   </div>
</div>
```

Sin herramientas de compilación complejas. Vue funciona directamente en el navegador.

El HTML final queda semántico, limpio y libre de lógica pesada de JavaScript.

NotebookLM

Todo Junto (Parte 2): La Lógica

app.js

```
const { createApp, ref, onMounted } = Vue;

const app = createApp({
  setup() {
    const quotesList = ref([]); 1
    onMounted(async () => {
      const res = await fetch('quotes.json'); 2
      quotesList.value = await res.json(); 3
    });
    return { quotesList }; 4
  }
});

app.component('quote-item', { 5
  props: { text: String, author: String },
  template: `
    <div class=\"card\">
    <p class=\"quote\">"{{ text }}"</p>
    <p class=\"author\">- {{ author }}</p>
  </div>`
});

app.mount('#app'); 6
```

1. Estado inicial.
2. Petición externa.
3. Actualización reactiva.
4. Exposición al HTML.
5. Registro del UI.
6. Arranque de la app.

NotebookLM

Estilos en Vue: Clases vs. Estilos Dinámicos

Estilos Estáticos (CSS Tradicional)

Manejamos el diseño base usando clases estándar en nuestro HTML/CSS.

CSS snippet

```
.card {
  background-color: #242424;
  border-left: 4px solid #42b883;
}
```

HTML snippet

```
<div class="card">...</div>
```

Estilos Dinámicos (`:style`)

Vue inyecta estilos directamente desde JavaScript usando la directiva `:style` (ideal cuando el diseño depende de los datos).

```
<!-- El color de la tarjeta viene del JSON (item.color) -->
<div class="card" :style="{ borderLeftColor: item.color }">
  <p>{{ text }}</p>
</div>
```

Nota que usamos `camelCase` (`borderLeftColor`) en lugar de `kebab-case` dentro del objeto JS.

NotebookLM

Tu Kit de Herramientas Vue 3 (Resumen)

📦 Componentes

(`app.component`) Tu bloque de construcción Lego. Agrupa UI y lógica en etiquetas personalizadas.

🧩 Props

El pasaporte de los datos. Permite la comunicación estrictamente unidireccional (Padre ➜ Hijo).

🔁 v-for

Tu máquina clonadora. Itera sobre arrays para crear múltiples elementos (¡Nunca olvides usar `:key`!).

⏱️ onMounted

El despertador de la app. Ejecuta código, como peticiones `fetch`, apenas el componente nace en la pantalla.

⚡ ref()

La magia reactiva. Una variable que avisa automáticamente al HTML cuando su `.value` es modificado.

🎨 :style

CSS manejado por JavaScript. Permite modificar propiedades visuales en tiempo real según el estado de los datos.

Dominar estos 6 conceptos te dará el control sobre el 80% del flujo de trabajo diario en Vue 3.

NotebookLM